Contenido

1.CREACION DE TABLAS 2

1.1 TIPOS DE DATOS MYSQL 5

1.2 RESTRICCIONES DE INTEGRIDAD (CONSTRAINT = RESTRICCIÓN) 6

2. MODIFICACION DE LA ESTRUCTURA DE UNA TABLA 12

2.1 Añadir nuevas columnas 12

2.2 Modificar la definición de una columna 13

2.3 Restricciones al modificar la estructura de una tabla 14

3. AÑADIR COMENTARIOS 14

5. CAMBIO DEL NOMBRE A UNA TABLA o COLUMNA 15

6. BORRADO DE TABLAS 16

9. EJERCICIOS PROPUESTOS 17

1.CREACION DE TABLAS

Para crear una tabla en la base de datos se utiliza el comando **CREATE TABLE**. Este comando insertará la definición de la tabla en el diccionario de datos y reservará espacio en los ficheros de la base de datos para la introducción de la información.

En el comando se deberá indicar:

- el nombre de la tabla
- por cada columna de la tabla
  - * el nombre de la columna
  - * el tipo de datos

    (CHAR, VARCHAR, INT, DATE, LONGTEXT)
  - * el tamaño máximo de las columnas de tipo numérico y alfanumérico (sólo es obligatorio para las de tipo alfanumérico)
  - * si se permite que la columna contenga valores nulos

Las anteriores características se almacenan en las tablas del diccionario de datos para comprobaciones posteriores.

El comando CREATE TABLE se puede utilizar con dos formatos distintos.

En el primer formato tenemos que definir una por una todas las columnas de la tabla.

```
CREATE TABLE  <nombre_de_la_tabla>

(<nombre_de_la_columna1> <tipo_de_datos> [(tamaño)] [NOT NULL],
 <nombre_de_la_columna2> <tipo_de_datos> [(tamaño)] [NOT NULL],
...
<nombre_de_la_columna254> <tipo_de_datos> [(tamaño)] [NOT NULL]);
```

Ejemplo:

```
SQL> CREATE TABLE libros (
2        título   VARCHAR(40) PRIMARY KEY,
3        fecha_de_publicación DATE NOT NULL,
4       precio   INT,
5        resumen LONGTEXT,
6        autor  VARCHAR(40)
7        );
```

Table created.

Se puede comprobar que la tabla no contiene ninguna fila.

```
SQL>SELECT * FROM libros;
```

no records selected.

Con el segundo formato se copia la definición de tablas ya existentes en el diccionario de datos, y además se copian también las filas de esas tablas.

CREATE TABLE <nombre_de_la_tabla> [< lista_nuevas_columnas >]

AS SELECT < lista_de_columnas_existentes >

FROM < lista_de_tablas_existentes >

WHERE < lista_de_condiciones >;

< lista_nuevas_columnas >

Nombre de las nuevas columnas de la tabla separados por comas.

< lista_de_columnas_existentes >

Nombre de columnas de tablas que aparecen en el FROM.

< lista_de_tablas_existentes >

Nombre de las tablas de las que se va a copiar la definición de la columnas.

< lista_de_condiciones >

Condiciones que deben cumplir las filas que se van a copiar.

Ejemplo:

```
SQL> CREATE TABLE libros_baratos
2       AS SELECT título, precio, fecha_de_publicación
3       FROM libros
4       WHERE precio <= 500;
```

Table created.

Nombres de tablas y columnas

NOMBRES UNICOS

- los nombres de las tablas de un usuario no se pueden repetir
- los nombres no pueden coincidir con palabras clave de MYSQL
- los nombres de las columnas deben ser únicos dentro de una tabla

CARACTERES

- el primer carácter de un nombre debe ser una letra mayúscula o minúscula
- el resto de los caracteres es libre con la excepción de las comas
- los nombres pueden tener como máximo 30 caracteres
- si se incluye el nombre entre dobles comillas no se tienen por qué tener en cuenta las restricciones anteriores

1.1 TIPOS DE DATOS MYSQL

Los datos de la base de datos MYSQL se almacenan en las filas de las tablas. Una tabla puede contener hasta 254 columnas y cada columna puede ser de uno de los siguientes tipos de datos:

- VARCHAR
- INT
- DATE
- LONGTEXT

Tipo de datos alfanumérico

Se usan para guardar palabras, frases o datos alfanuméricos:

- CHAR(M): Longitud fija (de 0 a 255 caracteres). Rellena los espacios sobrantes.
- VARCHAR(M): Longitud variable (hasta 65,535 bytes). Solo ocupa el espacio del texto más 1 byte adicional.
- TINYTEXT / TEXT / MEDIUMTEXT / LONGTEXT: Para textos largos, desde 255 caracteres hasta varios gigabytes.
- BINARY y VARBINARY: Similares a CHAR y VARCHAR, pero almacenan bytes en lugar de caracteres de texto.
- BLOB (TINYBLOB, BLOB, MEDIUMBLOB, LONGBLOB): Para datos binarios pesados como imágenes o archivos.
- ENUM: Lista cerrada de valores predefinidos (máximo 65,535 opciones).
- SET: Permite seleccionar múltiples valores de una lista definida (hasta 64 elementos).

Tipo de datos numérico

Se usan para almacenar números enteros o con decimales:

- **Enteros:**
  - TINYINT: 1 byte (de -128 a 127, o 0 a 255 sin signo). (Usado para booleanos como TINYINT(1)).
  - SMALLINT: 2 bytes (de -32,768 a 32,767).
  - MEDIUMINT: 3 bytes.

- INT / INTEGER: 4 bytes (±2.147 millones).
- BIGINT: 8 bytes para números muy grandes.

- **Decimales y de precisión:**
  - FLOAT: 4 bytes, punto flotante de precisión simple.
  - DOUBLE: 8 bytes, punto flotante de precisión doble.
  - DECIMAL(M, D) / NUMERIC: Precisión exacta, ideal para dinero.

Tipo de datos fecha

Se utilizan para registrar momentos temporales:

- DATE: Almacena solo la fecha (Formato: YYYY-MM-DD).
- TIME: Almacena solo la hora (Formato: HH:MM:SS).
- DATETIME: Combina fecha y hora (Abarca desde el año 1000 hasta el 9999, ocupa 8 bytes).
- TIMESTAMP: Marca de tiempo vinculada a UTC (Abarca de 1970 a 2038, ocupa 4 bytes).
- YEAR: Almacena un año de 2 o 4 dígitos (1901 a 2155).

Tipo de datos ESPACIALES

Orientados a sistemas de información geográfica y geometrías:

- GEOMETRY, POINT, LINESTRING, POLYGON
- Colecciones como MULTIPOINT, MULTILINESTRING, MULTIPOLYGON y GEOMETRYCOLLECTION

Tipo JSON

JSON: Permite almacenar documentos estructurados en formato JSON de forma nativa con validación automática de sintaxis.

1.2 RESTRICCIONES DE INTEGRIDAD (CONSTRAINT = RESTRICCIÓN)

La versión 6 de MYSQL admite en la sintaxis de las sentencias SQL CREATE TABLE y ALTER TABLE la especificación de restricciones de integridad de tabla y de columna. Estas restricciones se introducen en las tablas del diccionario de datos para que se verifiquen en posteriores versiones de MYSQL, puesto que en la versión actual no se comprueban (a excepción de NOT NULL).

Las restricciones se deben comprobar al ejecutar cualquier consulta de manipulación de datos y se deben cumplir para que las sentencias DML tengan éxito.

Las restricciones de tabla forman parte de la definición global de la tabla, se especifican después de la definición de las columnas y se pueden referir a una o más columnas de la tabla:

```
[ { UNIQUE | PRIMARY KEY } ( <lista_de_columnas> )

[CONSTRAINT <nombre_restricción>] ]        [ FOREIGN KEY ( <lista_de_columnas> )

REFERENCES [<base_de_datos>.]<tabla> [ (lista_de_columnas) ]

[CONSTRAINT<nombre_restricción>]][CHECK(<condición>)[CONSTRAINT<nombre_restricción>] ]
```

Las restricciones de integridad **de columna** se refieren a la columna en cuya descripción se encuentra la restricción:

```
[ { NULL | NOT NULL } [CONSTRAINT <nombre_restricción>] ]
[ { UNIQUE | PRIMARY KEY }
[CONSTRAINT <nombre_restricción>] ]
[ FOREIGN KEY REFERENCES [<usuario>.]<tabla> [(columna)]
  [CONSTRAINT <nombre_restricción>] ]
[ CHECK (<condición>) [CONSTRAINT <nombre_restricción>] ]
```

Si a una restricción no se le da nombre el sistema se encarga de dárselo con el siguiente formato: SYS_Cn, donde la n es un número para garantizar la unicidad de los nombres de restricciones.

Tipos de restricciones, **CONSTRAINT**:

NULL: Indica si una columna puede ser o no nula en una tabla. Not null indica **prohibir nulos.**

```
CREATE TABLE cliente(
        dni VARCHAR2(9) NOT NULL);
```

(el nombre lo coloca mysql)

```
CREATE TABLE cliente
 (dni VARCHAR2(9)
   CONSTRAINT cliente_dni_sinnulos  NOT NULL);
```

(le ponemos nombre a la restricción nosotros)

UNIQUE: Indica que todas las filas de una tabla deben tener un valor distinto para esa columna o columnas que deben ser NOT NULL y no pueden ser PRIMARY KEY. Los **valores únicos** son las claves candidatas de la tabla y se pueden declarar también de dos formas.

```
CREATE TABLE cliente(dni VARCHAR2(9) UNIQUE) ;
```

(el nombre lo coloca mysql)

```
CREATE TABLE cliente(dni VARCHAR2(9)
CONSTRAINT cliente_dni_u  UNIQUE);
```

(le ponemos nombre a la restricción nosotros)

PRIMARY KEY: Indica que esa columna o columnas constituyen la **clave primaria** de la relación y digamos que lleva implícito que sea NOT NULL y UNIQUE. Se puede especificar sólo una vez en cada tabla. Si la clave primaria es una única columna se puede especificar como restricción de tabla o de columna, si la clave primaria la forman más de una columna se puede especificar sólo como restricción de tabla.

```
CREATE TABLE cliente(dni VARCHAR2(9) PRIMARY KEY);
```

(el nombre lo coloca mysql)

```
CREATE TABLE cliente(dni VARCHAR2(9)
CONSTRAINT cliente_dni_PK  PRIMARY KEY);
```

(le ponemos nombre a la restricción nosotros)

FOREIGN KEY/ REFERENCES: Indica que esa columna o columnas son una clave ajena de la columna o columnas que son clave primaria de la tabla que se indica detrás de la cláusula REFERENCES. **Clave secundaria o foránea** y no va a permitir inserciones ni actualizaciones si el valor correspondiente no existe ya en la otra tabla. **No va a permitir borrados que violen la restricción de integridad.**

```
CREATE TABLE alquiler(
        dni VARCHAR2(9),
        cod_pelicula NUMBER(5),
        CONSTRAINT alquiler_pk   PRIMARY KEY(dni,cod_pelicula),
        CONSTRAINT dni_fk        FOREIGN KEY (dni)
                REFERENCES clientes(dni),
        CONSTRAINT pelicula_fk   FOREIGN KEY (cod_pelicula)
         REFERENCES peliculas(cod));
```

Esta completa forma de crear la tabla alquiler incluye sus claves foráneas, el campo *dni* hace referencia al campo dni de la tabla clientes y el campo *cod_pelicula* que hace referencia al campo *cod* de la tabla *peliculas*. También hubiera bastado con indicar sólola tabla a la que hacemos referencia, si no se indican los campos relacionados de esa tabla, se toma su clave principal (que es lo normal).

Importante mantener una **nomenclatura que indique que tabla y que tipo de índice o restricción tiene ese campo, _FK , _PK, _IX, etc.**

La integridad referencial es una herramienta imprescindible de las bases de datos relacionales. Pero provoca varios problemas. Por ejemplo, si borramos un registro en la tabla principal que está relacionado con uno o varios de la secundaria ocurrirá un error, ya que de permitírsenos borrar el registro ocurrirá fallo de integridad (habrá claves secundarios refiriéndose a una clave principal que ya no existe).

Por ello Mysql nos ofrece dos soluciones a añadir tras la cláusula REFERENCES:

ON DELETE SET NULL. Coloca nulos todas las claves secundarias relacionadas con la borrada.

ON DELETE CASCADE. Borra todos los registros cuya clave secundaria es igual que la clave del registro borrado.

Si no se indica esta cláusula, no se permite el borrado de registros relacionados.

El otro problema ocurre si se desea cambiar el valor de la clave principal en un registro relacionado con claves secundarias. En muchas bases de datos se implementan soluciones consistentes en añadir ON UPDATE CASCADE o ON UPDATE SET NULL. Mysql no implementa directamente estas soluciones.

La sintaxis completa para añadir claves foráneas es:

```
CREATE TABLE tabla(lista_de_campos
CONSTRAINT nombreRestriccion FOREIGN KEY (listaCampos)
REFERENCES tabla(clavePrincipalRelacionada)
[ON UPDATE {SET NULL | CASCADE}]      );
```

Si es de un solo campo existe esta alternativa:

```
CREATE TABLE tabla(lista_de_campos tipos propiedades,
nombreCampoClaveSecundaria
CONSTRAINT nombreRestriccion
REFERENCES tabla(clavePrincipalRelacionada)
[ON UPDATE {SET NULL | CASCADE}]
);
```

- **CHECK:**

  Indica que los valores de una columna o un grupo de columnas de una fila deben cumplir la condición antes de que se pueda insertar o actualizar esa fila.

```
CONSTRAINT chk_edad CHECK (edad >= 18);
CONSTRAINT ch_sexo CHECK (SEXO in ('hombre', 'mujer', 'intersexual'));
```

- **AUTO_INCREMENT:**

  En MySQL. Se utiliza para generar automáticamente un número secuencial único cada vez que se inserta un nuevo registro en una tabla.

  Es la forma estándar en MySQL para crear identificadores o claves primarias automáticas (como el número de ID de un usuario, cliente o factura).

```
CREATE TABLE productos (
 id INT AUTO_INCREMENT,
nombre VARCHAR(100) NOT NULL,
PRIMARY KEY (id)
 );
```

2. MODIFICACION DE LA ESTRUCTURA DE UNA TABLA

Una de las principales características de MYSQL es la posibilidad que ofrece de modificar la definición de las tablas después de haber sido creadas.

Para ello se dispone del comando ALTER TABLE.

2.1 Añadir nuevas columnas

Para añadir nuevas columnas a la definición de una tabla el comando ALTER TABLE tiene el siguiente formato:

```
ALTER TABLE  <nombre_de_la_tabla>
ADD  ( <nombre_de_la_nueva_columna1>  <tipo_de_datos>,
          <nombre_de_la_nueva_columna2>  <tipo_de_datos>,
          ... );
```

Ejemplo:

```
ALTER TABLE libros
        ADD   ( género    VARCHAR2(30),  país         VARCHAR2(10)
);
```

No se pueden especificar las columnas como no nulas ya que necesariamente contendrán valores nulos en un principio.

2.2 Modificar la definición de una columna

Para modificar la definición de una columna utilizaremos el siguiente formato del comando ALTER TABLE:

```
ALTER TABLE  <nombre_de_la_tabla>
MODIFY  ( <nombre_de_la_columna1> <tipo_de_datos> [NULL/NOT NULL],
      <nombre_de_la_columna2> <tipo_de_datos> [NULL/NOT NULL],
                ... );
```

Ejemplo:

```
SQL>ALTER TABLE  libros
        MODIFY  ( título CHAR(100) );
```

2.3 Restricciones al modificar la estructura de una tabla

Se podrá siempre:

- cambiar la definición de una columna para permitir valores nulos
- aumentar el tamaño de definición de una columna
- añadir una columna que permita valores nulos

Si en una tabla no hay filas que tengan valores en la columna que van a ser modificada, se podrá:

- disminuir el tamaño de la columna
- cambiar el tipo de dato de la columna

Si en una tabla todas las filas tienen valores no nulos en la columna que va a ser modificada, se podrá:

- cambiar la definición de la columna para que no pueda contener valores nulos

Si una tabla está vacía, se podrá:

- añadir una columna que no admita nulos (se añade primero la columna y después se modifica para que no admita valores nulos)

3. AÑADIR COMENTARIOS

Para incluir comentarios sobre tablas y columnas en el diccionario de datos se utiliza el comando **COMMENT.**

Ejemplo comentario en columnas:

```
CREATE TABLE empleados (
id INT AUTO_INCREMENT PRIMARY KEY,
salario DECIMAL(10,2) COMMENT 'Monto mensual bruto en euros',
estado TINYINT(1) COMMENT '0 = Inactivo, 1 = Activo'
);
```

El base_de_datos es el único autorizado en hacer comentarios sobre sus tablas y/o columnas.

El texto del comentario en columna puede tener 1024 caracteres como máximo.

Ejemplo comentario en columnas:

```
CREATE TABLE logs_sistema (
evento_id INT AUTO_INCREMENT PRIMARY KEY,
descripcion TEXT
) COMMENT = 'Tabla para registrar errores críticos del sistema y auditorías';
```

El texto del comentario en tabla puede tener 2048 caracteres como máximo.

5. CAMBIO DEL NOMBRE A UNA TABLA o COLUMNA

Para cambiar de nombre a una tabla sin que se vea afectado el almacenamiento de sus datos se utiliza el comando RENAME.

RENAME TABLE <antiguo_nombre> TO <nuevo_nombre>;

Ejemplo:

```
RENAME TABLE emp TO empleados;
```

En el diccionario de datos la tabla quedará reflejada con el nuevo nombre, pero no tendrá ninguna repercusión sobre el almacenamiento de los datos.

La tabla no se podrá volver a referenciar por el antiguo nombre.

El base_de_datos es el único que puede cambiar de nombre a una tabla.

Para cambiar de nombre a una columna sin que se vea afectado el almacenamiento de sus datos se utiliza el comando RENAME.

```
ALTER TABLE <nombre_tabla>
RENAME COLUMN <antiguo_nombre> TO <nuevo_nombre>;
```

Ejemplo:

```
ALTER TABLE clientes RENAME COLUMN telefono_fijo TO telefono;
```

6. BORRADO DE TABLAS

Para borrar la definición de las tablas del diccionario de datos se utiliza el comando DROP TABLE.

```
DROP TABLE <nombre_de_la_tabla>;
```

Cuando se ejecuta este comando se producen los siguientes efectos:

- la definición de la tabla será borrada del diccionario de datos
- el espacio que ocupaba la tabla se recupera para nuevos datos de la base de datos
- se borrarán todas las filas de la tabla sin previa advertencia
- no se podrán recuperar ni la definición ni los datos de la tabla
- una tabla sólo la puede borrar su base_de_datos

Si borramos la tabla que habíamos creado con anterioridad:

```
DROP TABLE libros_baratos;
```

Table dropped.

A continuación, intentamos consultar esa tabla:

```
SQL>SELECT * FROM libros_baratos;

ERROR at line 1: table or view does not exist
```

Diferencia entre la sentencia DROP TABLE y DELETE

La diferencia entre la sentencia DROP TABLE y la sentencia DELETE consiste en que la sentencia DROP TABLE además de borrar las filas de la tabla, también borra su definición en el diccionario de datos.

La sentencia DROP TABLE, por ser un comando del lenguaje de definición de datos se valida de forma automática por lo que no se podrá volver al estado anterior mediante un "rollback".

La sentencia DELETE borra todas las filas de una tabla si no imponemos ninguna condición, pero al ser un comando de manipulación de datos se puede volver la situación anterior si no se han validado los cambios.

9. EJERCICIOS PROPUESTOS

1º. Crear las siguientes tablas de acuerdo con las restricciones que se mencionan:

Tabla TIENDAS

Campo	tipo	long	NULOS?
NIF	ALFANUM	10	NO ADMITE
NOMBRE	ALFANUM	20	
DIRECCION	ALFANUM	20	
POBLACION	ALFANUM	20	
PROVINCIA	ALFANUM	20	

COD_POSTAL	NUMERICO	5

- La clave primaria es NIF.
- PROVINCIA ha de almacenarse en mayúsculas.

Tabla FABRICANTES

Campo	tipo	long	NULOS?
COD_FABRICANTE	NUMERICO	3	NO ADMITE
NOMBRE	ALFANUM	15	
PAIS	ALFANUM	15	

- La clave primaria es COD_FABRICANTE.
- Las columnas NOMBRE y PAIS han de almacenarse en mayúsculas.

Tabla ARTICULOS

Campo	tipo	long	NULOS?
ARTICULO	ALFANUM	20	NO ADMITE
COD_FABRICANTE	NUMERICO	3	NO ADMITE
PESO	NUMERICO	3	NO ADMITE
CATEGORIA	ALFANUM	10	NO ADMITE
PRECIO_VENTA	NUMERICO	4	
PRECIO_COSTO	NUMERICO	4	
EXISTENCIAS	NUMERICO	5	

- La clave primaria está formada por las columnas ARTICULO, COD_FABRICANTE, PESO y CAREGORIA.
- COD_FABRICANTE es clave ajena que refencia a la tabla FABRICANTES.

- PRECIO_VENTA, PRECIO_COSTO y PESO han de ser >0.
- CATEGORIA ha de ser 'primera', 'segunda' o 'tercera'.

Tabla PEDIDOS

Campo	tipo	long	NULOS?
NIF	ALFANUM	10	NO ADMITE
ARTICULO	ALFANUM	20	NO ADMITE
COD_FABRICANTE	NUMERICO	3	NO ADMITE
PESO	NUMERICO	3	NO ADMITE
CATEGORIA	ALFANUM	10	NO ADMITE
FECHA_PEDIDO	FECHA		NO ADMITE
UNIDADES_PEDIDAS	NUMERICO	4	NO ADMITE

- La clave primaria está formada por las columnas NIF, ARTICULO, COD_FABRICANTE, PESO, CATEGORIA Y FECHA_PEDIDO.
- COD_FABRICANTE es clave ajena que referencia a la tabla FABRICANTES.
- Las columnas ARTICULO, COD_FABRICANTE, PESO y CATEGORIA son clave ajena y referencian a la tabla ARTICULOS. Realizar un borrado en cascada.
- NIF es clave ajena y referencia a la tabla TIENDAS.
- UNIDADES_PEDIDAS ha de ser >0.
- CATEGORIA ha de ser 'primera', 'segunda' o 'tercera'.

Tabla VENTAS

Campo	tipo	long	NULOS?
NIF	ALFANUM	10	NO ADMITE
ARTICULO	ALFANUM	20	NO ADMITE
COD_FABRICANTE	NUMERICO	3	NO ADMITE
PESO	NUMERICO	3	NO ADMITE
CATEGORIA	ALFANUM	10	NO ADMITE

FECHA_VENTA	FECHA		NO ADMITE
UNIDADES_VENDIDAS	NUMERICO	4	

- La clave primaria está formada por las columnas NIF, ARTICULO, COD_FABRICANTE, PESO, CATEGORIA, y FECHA_VENTA.
- COD_FABRICANTE es clave ajena que referencia a la tabla FABRICANTES.
- Las columnas ARTICULO, COD_FABRICANTE, PESO Y CATEGORIA son clave ajena y referencian a la tabla ARTICULOS. Realizar un borrado en cascada.
- NIF es clave ajena y referencia a la tabla TIENDAS.
- UNIDADES_VENDIDAS ha de ser >0.
- CATEGORIA ha de ser 'primera', 'segunda' o 'tercera'.

2º. Añadir una restriccion a la tabla TIENDAS para que el NOMBRE de la tienda sea del tipo titulo.

3º. Visualizar las constraints definidas para las tablas anteriores.

4º. Modificar las columnas de las tablas PEDIDOS y VENTAS para que las UNIDADES_VENDIDAS y las UNIDADES_PEDIDAS puedan almacenar cantidades numéricas de 6 dígitos.

5º. Impedir que se den de alta mas tiendas en la provincia de 'TOLEDO'.

6º. Añadir a las tablas PEDIDOS y VENTAS una nueva columna para que almacenen el PVP del articulo.

7º. Añadir el siguiente comentario a la tabla VENTAS: Unidades vendidas de los distintos artículos de los fabricantes.

8º. Añadir el siguiente comentario al campo PESO de la tabla PEDIDOS 'peso del articulo pedido'.

9º. Crear un sinonimo llamado PED para la tabla PEDIDOS

10º. Cambiar de nombre a la tabla VENTAS y llamarla NUEVAS_VENTAS.

11º. Borrar la tabla Tiendas.

12º. Cree el siguiente modelo de datos:

SOCIOS

Cod_socio	nombre	apellidos	domicilio	poblacion	Cod_postal	provincia	telefono	sexo	observaciones
10300106	Santiago	Blanco Ruiz	Balmes 36	Llodio	01400	Álava	4531221	H	Todos los meses compra dos pelis
11369214	Carlos	Flores Vargas	Potosí 13	Alcorcon	28921	Madrid	5436785	H	
17269813	Ana	Lopez Pintado	Mayor 43	Ulldecona	43550	Tarragona	897654	M	Devuelve las pelis sin rebobinar

TEMAS

Cod_tema	tema
1	Suspense
2	terror
3	Histórica
4	Drama
5	Ciencia-ficcion

PELICULAS

Cod_pelicula	titulo	Interprete_1	Interprete_2
1	El color púrpura	Danny Glover	Whoopi Goldberg
2	A Proposito de Henry	Harrison Ford	Anette Bening
3	J.F.K.	Kevin Costner	Sissy Spacek

Director	productora	Cod_tema	Año_produccion	duracion	Precio_coste
Steven Spielberg	Warner Bros	5	1985	148	18
Mike Nichols	Columbia Pictures	4	1991	108	15
Oliver Stone	Warner Bros	3	1991	189	20

ALQUILER

Cod_alquiler	Cod_socio	Cod_pelicula	Fecha_alquiler	Fecha_devolucion	devuelta	precio
1	10300106	1	12/10/1998	18/10/1998	si	5
2	11369214	2	09/09/2000	15/09/2000	no	6
3	17269813	3	23/04/2002	01/05/2002	si	6
4	10300106	2	16/07/2004	25/07/2004	no	6
5	11369214	3	15/03/2006	18/03/2006	si	7
6	17269813	1	08/01/2007	15/01/2007	si	7

Añadiendo las Constraint que no sean de primary key, foreign key o no admitir nulos después de crear las tablas.

13º. Modificar las columnas de las tablas TEMAS y PELICULAS para que las Cod_tema puedan almacenar cantidades numéricas de 4 dígitos.

14º. Nos mosqueamos con el proveedor de películas de la productora Warner Bros. Impedir que se den de alta más películas de dicha productora en la tabla películas.

15º. Añadir el siguiente comentario a la tabla PELICULAS: “Peliculas producidas a partir de la segunda mitad del siglo XX”.

16º. Añadir el siguiente comentario al campo interprete_1 de la tabla PELICULAS: ‘actor principal’.

17º. Crear un sinónimo llamado ALQ para la tabla ALQUILER.

18º. Cambiar de nombre a la tabla SOCIOS y llamarla SOCIOS_NUEVOS.

19º. Borrar la tabla PELICULAS.